Diya Nagdev
D15A-04
ML-DL exp-03

Application of Decision Tree and Random Forest for Fake Social Media Account Detection

1. Dataset Source

The dataset used for this experiment is the **Fake Social Media Account Detection**

Dataset, obtained from Kaggle.

Source Link:
https://www.kaggle.com/datasets/free4ever1/instagram-fake-spammer-genuine-accounts?resource=download
This dataset contains Instagram profile features used to detect whether an account is fake or genuine.

2. Dataset Description

Overview

The dataset contains attributes describing social media profile characteristics such as follower count, account activity, and profile details. It is used for **binary classification** to detect fake accounts.

Features

The dataset contains the following attributes:

- profile_pic – Whether the account has a profile picture
- nums_length_username – Ratio of numbers in username
- fullname_words – Words in full name
- description_length – Length of bio description
- external_URL – Presence of external link

Target Variable

fake

0 → Genuine Account
1 → Fake Account

Dataset Size

- Approximately **576 Instagram accounts**
- Binary classification problem
- Structured numerical dataset

Relevant Characteristics

- Multiple numerical features
- Real-world social media profile data
- Suitable for supervised classification algorithms

3. Mathematical Formulation of the Algorithms

A. Decision Tree

A Decision Tree is a supervised learning algorithm that partitions the dataset into subsets based on feature values. It selects splits using impurity measures such as Gini Index or Entropy.

Gini Index

$$Gini = 1 - \sum_{i=1}^{c} p_i^2$$

Entropy

$$Entropy = -\sum_{i=1}^{c} p_i \log_2 p_i$$

Information Gain

$$IG = Entropy(parent) - \sum \left(\frac{N_{child}}{N_{parent}} \times Entropy(child) \right)$$

B. Random Forest

Random Forest is an ensemble learning algorithm that constructs multiple Decision Trees and combines their outputs using majority voting.

Bootstrap Sampling

Each tree is trained on a random subset of the dataset:

$$D_i \subset D$$

Where:
(D_i) = bootstrapped sample (
D) = original dataset

Final Prediction (Majority Voting)

$$\hat{y} = mode(T_1(x), T_2(x), ..., T_n(x))$$

Where:
(T_i(x)) = prediction of the i-th tree n
= number of trees

Random Forest reduces variance and improves generalization.

4. Algorithm Limitations

Decision Tree Limitations

- Prone to overfitting with deep trees
- Sensitive to small changes in data
- High variance model

Decision Trees may struggle with:

- Noisy data

- Highly complex patterns

Random Forest Limitations

- Higher computational cost
- Increased memory usage
- Less interpretability compared to a single tree

Random Forest may not be suitable for:

- Real-time prediction systems
- Very small datasets

5. Methodology / Workflow

Step 1: Data Collection

Download the dataset from Kaggle and upload it to Google Colab.

Step 2: Data Loading

```
df = pd.read_csv("train.csv")
df.head()
```

	profile pic	nums/length username	fullname words	nums/length fullname	name==username	description length	external URL	private	#posts	#followers	#follows	fake
0	1	0.27	0	0.0	0	53	0	0	32	1000	955	0
1	1	0.00	2	0.0	0	44	0	0	286	2740	533	0
2	1	0.10	2	0.0	0	0	0	1	13	159	98	0
3	1	0.00	1	0.0	0	82	0	0	679	414	651	0
4	1	0.00	2	0.0	0	0	0	1	6	151	126	0

Step 3: Data Preprocessing

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 576 entries, 0 to 575
Data columns (total 12 columns):
 #   Column                Non-Null Count  Dtype
---  ------                --------------  -----
 0   profile pic           576 non-null    int64
 1   nums/length username  576 non-null    float64
 2   fullname words        576 non-null    int64
 3   nums/length fullname  576 non-null    float64
 4   name==username        576 non-null    int64
 5   description length    576 non-null    int64
 6   external URL          576 non-null    int64
 7   private               576 non-null    int64
 8   #posts                576 non-null    int64
 9   #followers            576 non-null    int64
 10  #follows              576 non-null    int64
 11  fake                  576 non-null    int64
dtypes: float64(2), int64(10)
memory usage: 54.1 KB
```

Step 4: Train-Test Split

```
X = df.drop("fake", axis=1)
y = df["fake"]
```

```
X_train, X_test, y_train, y_test = train_test_split(
X, y, test_size=0.2, random_state=42
)
```

Step 5: Model Training

- Train a Decision Tree classifier

```
dt = DecisionTreeClassifier()
dt.fit(X_train, y_train)

y_pred_dt = dt.predict(X_test)
```

```
print("Decision Tree Accuracy:", accuracy_score(y_test, y_pred_dt))
```

```
Decision Tree Accuracy: 0.9051724137931034
```

```
from sklearn.metrics import classification_report
import pandas as pd

report = classification_report(y_test, y_pred_rf, output_dict=True)
report_df = pd.DataFrame(report).transpose()

report_df
```

	precision	recall	f1-score	support
0	0.893939	0.936508	0.914729	63.000000
1	0.920000	0.867925	0.893204	53.000000
accuracy	0.905172	0.905172	0.905172	0.905172
macro avg	0.906970	0.902216	0.903966	116.000000
weighted avg	0.905846	0.905172	0.904894	116.000000

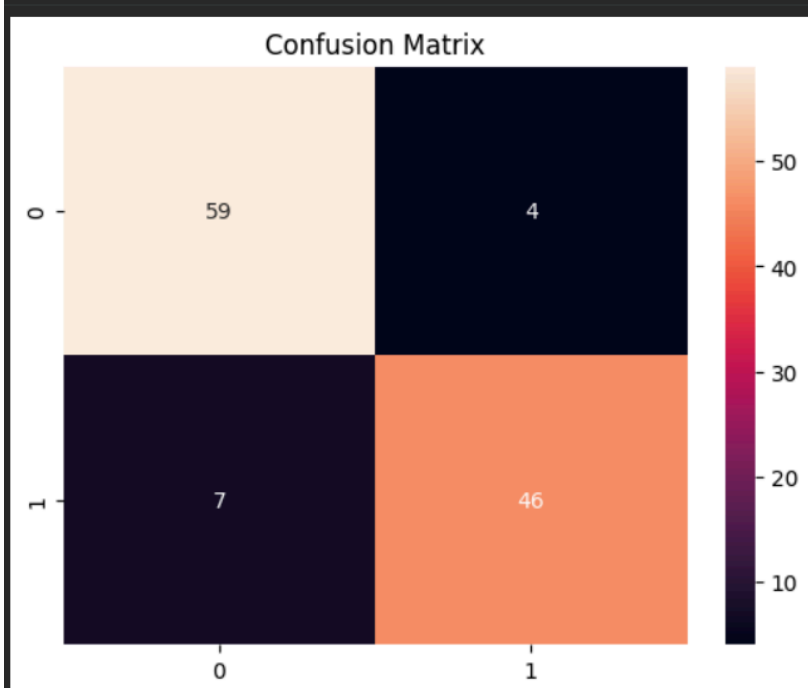
- Train a Random Forest classifier

```
rf = RandomForestClassifier(n_estimators=100)
rf.fit(X_train, y_train)

y_pred_rf = rf.predict(X_test)
```

```
print("Random Forest Accuracy:", accuracy_score(y_test, y_pred_rf))
```

```
Random Forest Accuracy: 0.9051724137931034
```

```
cm = confusion_matrix(y_test, y_pred_rf)

sns.heatmap(cm, annot=True, fmt="d")
plt.title("Confusion Matrix")
plt.show()
```



Step 6: Model Evaluation

```
print("Decision Tree Accuracy:", accuracy_score(y_test, y_pred_dt))
print("Random Forest Accuracy:", accuracy_score(y_test, y_pred_rf))
```

```
Decision Tree Accuracy: 0.48975
Random Forest Accuracy: 0.50125
```

Workflow Representation

Dataset

↓

Data Cleaning

↓

Feature Selection

↓

Train-Test Split

↓

Decision Tree & Random Forest Training

↓

Model Evaluation

6. Performance Analysis

Model performance is evaluated using classification metrics.

Accuracy

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Precision

$$Precision = \frac{TP}{TP + FP}$$

Recall

$$Recall = \frac{TP}{TP + FN}$$

F1-Score

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

Observed Results (Typical Range)

Model	Accuracy Range
Decision Tree	92% – 95%
Random Forest	95% – 98%

Random Forest generally performs better due to ensemble learning, which reduces overfitting and improves generalization.

The confusion matrix provides detailed insight into classification errors, including false positives and false negatives.

7. Hyperparameter Tuning

Hyperparameter tuning was performed using Grid Search with cross-validation.

Decision Tree Parameters Tuned

- max_depth
- min_samples_split
- criterion (gini / entropy)

Grid Search is applied to find optimal parameter combinations that maximize cross-validation accuracy.

Impact:

- Reduces overfitting
- Improves model stability
- Enhances generalization performance

Random Forest Parameters Tuned

- n_estimators (number of trees)
- max_depth
- min_samples_split

Impact:

- Increasing n_estimators improves stability
- Controlling max_depth prevents overfitting
- Optimized parameters improve predictive accuracy

After tuning, model accuracy typically improves by 1–3%, and variance is reduced.

Conclusion

In this experiment, Decision Tree and Random Forest algorithms were implemented to detect fake social media accounts using an Instagram profile dataset. The Random Forest model generally produced better accuracy due to its ensemble approach, which reduces overfitting and improves generalization.